

Code Review Report: `CinematicBars.java`

Tanveer Muntaseer

March 31, 2026

Introduction

This report documents a manual code review of `CinematicBars.java` from the Spring 2026 Team 15 project repository. The class is responsible for rendering animated letterbox (cinematic) bars that slide in from the top and bottom edges of the game screen during cutscene sequences. It implements the `Disposable` interface from the libGDX framework and manages a four-state animation lifecycle: `HIDDEN`, `APPEARING`, `VISIBLE`, and `DISAPPEARING`. While the class is relatively compact and functional, the review identified five code smells that impact maintainability, testability, and long-term extensibility.

Code Smells Identified

1. High Coupling to `GameScreen` (Lines 74–75)

Issue: The `render()` method directly references static fields from the `GameScreen` class:

```
1 batch.draw(blackTexture, 0, 0, GameScreen.VIEW_WIDTH, currentHeight);
2 batch.draw(blackTexture, 0, GameScreen.VIEW_HEIGHT - currentHeight,
3           GameScreen.VIEW_WIDTH, currentHeight);
```

This creates a hard compile-time dependency between `CinematicBars` and `GameScreen`. As a result, `CinematicBars` cannot be instantiated or tested without `GameScreen` being on the classpath, violating the Dependency Inversion Principle. If the game later supports multiple screen resolutions or the viewport dimensions change, every class that hardcodes these references would need to be updated.

Refactoring: Inject screen dimensions through the constructor and store them as private instance fields. This decouples the class, enables unit testing with arbitrary dimensions, and makes the class reusable across different screen configurations:

```
1 private final float screenWidth;
2 private final float screenHeight;
3
4 public CinematicBars(float screenWidth, float screenHeight) {
5     this.screenWidth = screenWidth;
6     this.screenHeight = screenHeight;
7     // ... existing Pixmap setup ...
8 }
```

2. Redundant State Field (Lines 21–22, 37, 44)

Issue: The class maintains both `timer` and `currentHeight` as instance fields, but `currentHeight` is always mathematically derivable from `timer`, `state`, `BAR_HEIGHT`, and `ANIM_DURATION`. This redundancy forces the `show()` and `hide()` methods to perform inverse calculations to keep the two fields synchronized:

```
1 timer = (currentHeight / BAR_HEIGHT) * ANIM_DURATION;           // show()
2 timer = ANIM_DURATION * (1f - currentHeight / BAR_HEIGHT);      // hide()
```

Maintaining two representations of the same underlying value introduces the risk of them drifting out of sync, especially as the class evolves. This is a classic *data clump* smell where redundant state adds complexity without benefit.

Refactoring: Keep only `timer` as the authoritative field and compute `currentHeight` on demand inside `update()` and `render()` via a private helper method such as `computeHeight()`.

3. Missing Javadoc on Public Methods (Lines 34, 42, 49, 79, 84)

Issue: While the class-level Javadoc is informative, the five public methods — `show()`, `hide()`, `update(float delta)`, `isFullyVisible()`, and `dispose()` — lack any documentation. A developer integrating this class into a new screen or cutscene system would need to read the full implementation to understand method contracts, preconditions (e.g., `update()` must be called every frame), and threading expectations.

Refactoring: Add Javadoc to every public method describing its purpose, parameters, and any important side effects:

```
1 /** Begins the bar-appearing animation. If called mid-disappear,
2  * smoothly reverses from the current position. */
3 public void show() { ... }
4
5 /** Advances the animation state machine by the given time step.
6  * Must be called once per frame.
7  * @param delta elapsed time in seconds since the last frame */
8 public void update(float delta) { ... }
```

4. Abbreviated Variable Name (Line 27)

Issue: The local variable `pm` in the constructor is a non-descriptive abbreviation for `Pixmap`. While the scope is small, abbreviated names reduce readability and violate standard Java naming conventions, which favour descriptive identifiers. A future maintainer scanning the constructor must mentally map the abbreviation to its type.

Refactoring: Rename to `pixmap` for immediate clarity. The change is trivial but contributes to overall code cleanliness and consistency across the codebase.

5. Implicit Else Branch Without Explicit Guard (Line 60)

Issue: In `update()`, the `else` branch assumes the state is `DISAPPEARING` based on the early return at line 50 filtering out `HIDDEN` and `VISIBLE`. A comment `// DISAPPEARING` documents this intent, but the logic is implicit and fragile. If a future developer adds a new state to the enum (e.g., `PAUSED`) and forgets to add an early return, the `else` branch would silently execute the disappearing logic for the wrong state, causing a subtle animation bug.

Refactoring: Replace the bare `else` with an explicit guard `else if (state == State.DISAPPEARING)`. This makes the code self-documenting, prevents accidental fall-through with new states, and eliminates the need for the clarifying comment entirely.

Summary

Five code smells were identified in `CinematicBars.java`: (1) tight coupling to `GameScreen` via static field access, (2) redundant synchronized state between `timer` and `currentHeight`, (3) missing Javadoc on all public methods, (4) a cryptic variable abbreviation, and (5) an implicit else branch that is fragile to future state additions. Each smell has a concrete refactoring path that improves the design without altering the class's external behaviour. All refactored changes should be verified by re-running the existing test suite after each modification.